

完整体 IP 协议

常青

(北京爱辟加阿加网络科技有限公司, 北京 100195)

Email:cq@ipp.pxyz

摘要: IPv4 采用 32 位定长二进制地址, 其地址空间不足的问题早已凸显。网络地址转换 (NAT) 作为临时应对方案, 虽在一定程度上缓解了地址稀缺压力, 却破坏了互联网端到端通信的核心原则, 仅支持内网主动访问外网, 无法满足外网对内网的直接访问需求。尽管业界陆续提出多种解决方案, 但均存在固有局限。本文提出一种基于多级变长地址的创新方案 (IP++) , 可从根源上破解这一难题, 使 IP 协议回归端到端的基本原则。该方案的多级变长地址结构与互联网天然的层次化拓扑高度契合, 具备兼容性强、部署便捷、易用性突出等核心优势。

关键词: 变长地址; 端到端; IP++; 封装; 翻译

中图分类号: TP393.08 **文献标志码:** A

Complete Internet Protocol

Qing Chang

Beijing IpPlusPlus Network Technology Company Limited, Beijing 100195, China

Email:cq@ipp.pxyz

Abstract: IPv4 adopts 32-bit fixed-length addresses, the problem of insufficient address space has long been prominent. Network Address Translation (NAT), as a temporary workaround, has alleviated the pressure of address scarcity to a certain extent, but undermined the core principle of end-to-end communication on the Internet. It only supports active access from the internal network to the external network and cannot meet the demand for direct access from the external network to the internal network. Although various solutions have been proposed successively in the industry, all of them suffer from inherent limitations. This paper proposes an innovative multi-level variable-length address scheme named IP++, which fundamentally solves this dilemma and enables the IP protocol to return to the basic end-to-end principle. The multi-level variable-length address structure of the proposed scheme is highly compatible with the natural hierarchical topology of the Internet, and it possesses core advantages such as strong compatibility, convenient deployment and

outstanding usability.

Key words: variable-length address; end-to-end;IP++; encapsulation; translation

1 问题的由来

网络技术体系并非一次性整体设计而成，而是逐步演化而来[1,2]。传统的 IPv4 采用 32 位地址[3]。随着互联网规模的不断增大，地址资源不足的问题越来越凸显。为缓解这一困境，业界引入了 NAT（网络地址转换）[4]，其本质是通过地址共享机制实现多台内网设备复用少量公网地址。但 NAT 破坏了端到端的基本原则，造成了外网无法直接访问内网设备，形成了远程互联的难题。

为解决远程互联的难题，业界先后提出多种技术路径，但均未能实现全场景完美适配，以下对主流方案的核心特征与局限展开详细分析：

1.1 服务器中转

服务器中转是目前应用最广泛的远程互联方案。依托公网服务器的全局可达性，内网两端设备通过接入同一中转服务器建立间接通信链路，实现数据互通。实际部署中，既可以自主开发中转服务，也可复用成熟中间件，例如以 frp 为代表的反向代理工具、MQTT 框架等。反向代理工具通过在中转服务器与内网设备间建立隧道映射，使用户访问中转服务器的指定端口时，流量被转发至内网目标设备，间接实现外网访问；MQTT 则借助现成的框架简化部署流程，降低开发成本。服务器中转的缺点比较明显，就是通信路径迂回，不仅会导致网络时延增加，还会衍生出开发、运维及安全等方面的一系列问题。

1.2 端口映射

端口映射通过在网关上配置规则，建立网关公网端口与内网设备端口的一一映射关系。当外网用户访问网关的指定端口时，数据包会被自动转发至内网对应设备的目标端口，从而实现外网对内网的访问。但 IP 地址与端口的功能定位存在本质差异：IP 地址属于网络层标识，用于定位网络中的终端节点；端口属于传输层标识，用于区分终端上的不同应用进程。依赖端口映射实现远程访问，必然造成网络层与传输层的功能错位，进而引发管理混乱。例如，端口的归属权通常属于终端设备所有者，而端口映射配置由网关管理者操作，权责分离易导致配置冲突；此外，同类应用在不同节点

上通常默认使用相同端口，若通过端口区分不同节点，会违背用户使用习惯，增加操作复杂度。因此，端口映射仅适用于特定简单场景，无法作为远程互联的通用解决方案。

1.3 VPN

VPN 通过在不同内网之间建立加密隧道，模拟“同一内网”的网络环境，实现跨地域内网资源的互联互通，在企业分支互联、远程办公等场景中应用广泛。该方案的核心优势在于安全性高、能复用现有内网资源，但使用场景存在显著局限：其前提是跨网两端需处于统一管理域（如同一企业、同一组织）。因此，VPN 仅能作为特定场景的专用技术，难以成为解决远程互联问题的基础性、普适性方案。

1.4 IPv6

既然远程互联难题的根源在于 IP 协议本身的设计缺陷，业界自然将目光投向下一代 IP 协议的研发，IPv6 应运而生[5-7]。IPv6 采用 128 位的长地址，能有效解决地址资源不足的问题。但由于其扁平化的地址结构与互联网固有的层次结构之间存在根本性矛盾，导致 IPv6 存在两大致命缺陷：一是兼容性差，与 IPv4 无法无缝衔接，存量 IPv4 设备、网络设施的升级改造成本极高，难以大规模推广；二是易用性不足，长地址的配置、管理、记忆难度远超 IPv4。而且进一步导致了，存在一些极不适用的领域，如内网等。这些缺陷注定 IPv6 难以完全取代 IPv4，无法成为统一的下一代 IP 协议。

2 基于变长地址的 IP++方案

通过前面的介绍，我们可以得出结论：必须寻求一种新的方案来解决远程互联的难题。这种方案应该是什么样的呢？在能解决远程互联问题的前提下，应该吸取 IPv6 的教训，必须兼容。这不仅因为庞大规模的存量设施意味着过渡问题必须认真对待，而且现有的网络总体上是成功的，多数时候能够胜任。因此新方案应该是扩展，而不是推倒重来。

细心观察，我们会发现身边存在大量分级变长标识的例子，如通信地址、电话号码、文件路径、域名等等。这些例子全都采用相同的处理方法，不可能是巧合，而应有内在的原因。这些例子所处理的对象都有层次聚合的特征。多级变长标识与层次聚合特征天然契合，是最自然、最高效的标识方式。现实世界中，由多个对象组成的群

组常常具有这种层次聚合的特征，因此我们身边有很多采用分级变长标识的例子也就并不奇怪了。网络节点也符合层次聚合的特征，因此互联网节点的编址也应采用多级变长结构。定长地址只是在初期节点数较少、组织结构较简单情况下的临时方案。IPv4 的“公网地址+私有地址”模式虽在一定程度上体现了多级变长的思路，但未能形成系统化的多级变长架构。IP++方案的核心思想是：将多级变长地址全面引入网络层协议，构建与网络层次拓扑完全匹配的编址体系，从根源上解决远程互联难题[8]。

需要特别指出的是，自 TCP/IP 协议体系问世以来，业界针对变长地址的探索从未间断。全面采用变长地址架构，始终是 IP 协议设计者的终极理想，而历代商用 IP 协议，均是技术理想与工程现实相互妥协后的产物。在 TCP/IP 协议初创阶段，变长地址方案便已纳入设计考量，只因当时硬件处理性能有限，最终未能付诸落地[9]。以 David D. Clark、Vinton Cerf、J. Noel Chiappa、Christian Huitema 为代表的互联网先驱，长期致力于变长地址理论研究与技术实践[10,11]。IPv6 标准化制定阶段，是 IP 协议采用变长地址的关键契机，业界就变长地址的取舍展开激烈争论[12,13]。遗憾的是，经多方博弈与折中妥协后，IPv6 仅在局部引入变长前缀设计，并未整体采用变长地址架构。随着 IPv6 规模化部署遭遇现实瓶颈，Christian Huitema[14]、华为 2012 实验室[15]、毛德操、钱华林、汪涛等不断提出用变长地址解决困局。而 IP++实现了变长地址理论的系统化与可工程化实施方案。

2.1 单元网划分

IP++下互联网被划分为若干级，不同级和同级的不同子网各自使用独立的编址空间。我们将这种使用独立编址空间的网络区间称作单元网。每个单元网都拥有一个相当于 IPv4 的编址空间。

互联网的基本结构是树形的。最上一层只有一个单元网，即公网，相当于树形的根。由公网向下逐级挂接子单元网，最终展开成树形。

单元网及其上设备有明确的级数。级数的表示有绝对级数和相对级数两种方式。绝对级数在前面加“/”前缀，相对级数在前面加“.”前缀。绝对级数用一非负整数表示，公网为/0 级，向下依次为/1 级、/2 级等。相对级数用一整数表示，当前级为.0 级，向上依次为.-1 级、.-2 级等。

2.2 地址表示方法

IP++的地址由各级地址串联而成，这是分级变长标识的普遍做法，跟我们处理通信地址、文件路径时一致，很好理解。

完整的地址还要在前面加上起始级数，如/1#192.168.1.10。IP++中增加了相对地址。相对地址的作用和用法可以参考文件系统中的相对路径来理解。我们来看相对地址的例子。.0#192.168.1.10 表示当前单元网内的 192.168.1.10 这个地址。.-

1#10.66.7.250 表示当前单元网的父单元网内的 10.66.7.250 这个地址。

2.3 包头格式设计

IP++协议的实现需定义对应的包头格式，IP++包头分为基本头和扩展头两部分，兼顾核心功能承载与扩展能力。

基本头是必选的，承载数据包传输的核心信息（如服务类型、生存时间等）。IP++基本头的显著特征是可变长度设计——由“头长度”字段指定具体长度。基本头实际长度= $8+8\times 2^n$ 字节，比如“头长度”值为 0，则头的实际长度是 $8+8\times 2^0=16$ 字节。

IP++地址相对于 IPv4 地址复杂一些，使用了多个字段来表示。这里结合具体的例子来介绍，比如目的地址是/#1.1.1.2-1.1.1.28，在包头中是如何表示的呢？首先要指定是绝对地址还是相对地址，这里是绝对地址，因此“目的地址类型”字段就是 0。然后要指定起始级数和地址长度，这两项在“目的地址参数”字段中指定，前 4 比特指定起始级数，这里是 0，后 4 比特指定末端深入了几级，这里深入了 1 级，相应地，地址长度是 8 字节。这 8 个字节的数据存放在“目的地址、源地址”字段的起始位置。源地址的数据接在它后面，也就是从第 9 个字节开始。“目的地址、源地址”字段是可变长度的，具体长度由“头长度”字段决定。

IPv4 通过“选项”字段实现差异化功能，但受限于长度限制、解析复杂等问题，实际应用率极低。IP++改用“扩展头”承载差异化功能（如路由优化、安全认证、流量控制等）。扩展头为可选字段，支持按需添加，且具备良好的可扩展性，可根据业务需求灵活定义新的扩展头类型，无需修改基本头结构。具体扩展头类型及格式可参考相关技术文档。

3 兼容性设计

高度的兼容性是 IP++ 的核心优势之一。支持 IP++ 的设备与纯 IPv4 设备混合部署，终端同时运行 IP++ socket 与 IPv4 socket，整个网络体系仍能良好地运行。IP++ 是对 IPv4 的扩展，而不是推倒重来，二者在地址格式、协议逻辑上具备强延续性：公网和各个私网基本上就可以划为一个单元网，每个单元网内遵循的规则与 IPv4 雷同；IP++ 地址是 IPv4 地址的超集（IPv4 地址可视为 IP++ 的 /0 级地址），存量 IPv4 网络无需重新编址即可升级至 IP++；新旧网络可无缝衔接，升级过程可根据实际需求分阶段、分区域逐步推进，存量设备与投资可长期发挥效益；相关技术人员无需重构知识体系，学习成本极低。

IP++ 中，实现兼容性的具体策略有封装、翻译、双栈等，具体如下：

3.1 封装

IP++ 数据包在传输过程中可能会遇到纯 IPv4 设备，如果直接将数据包传给它，会让它不知所措，结果肯定是没能正确处理。这时只需将数据包套上一层 IPv4 外壳，使它看上去是一个 IPv4 数据包即可。IPv4 包头里的地址取 IP++ 地址中的当前单元网片段或向上的网关。如果该数据包后来又遇到了 IP++ 设备，则可以将其解封装。

IP++ 中的封装与传统意义的封装有着本质的区别。IP++ 地址与 IPv4 地址间有固有的对应关系，封装与解封装过程无需配置，可自然、方便地完成。

需要强调的是，封装与解封装必须成对存在，在某处封装后必定随后在某个地方解封装。封装与解封装对在某一个单元网内，数据包在跨越单元网时必定处于解封装状态。

3.2 翻译

翻译指 IP++ 数据包与 IPv4 数据包间的直接转换，其前提是地址长度匹配——即源地址与目的地址处于同一单元网内，此时 IP++ 可使用短地址格式（形如 `#x.x.x.x`），便于直接转换。若源地址与目的地址不在同一单元网（地址长度不匹配），则需结合 NAT 机制实现翻译。

传统 NAT 的核心目的是解决 IPv4 地址稀缺问题。IP++ 下，地址不再稀缺，单纯的 NAT 通常意义不大。IP++ 下使用 NAT 通常是配合翻译实现跨单元网的协议转换。IP++

中的 NAT 原理与传统 NAT 类似：当数据包进入目标单元网（末单元网）时，网际路由器将数据包的源地址替换为该单元网的网关地址，并记录地址映射状态；当反向数据包返回时，再将网关地址替换为原始源地址，实现双向通信。

IP++中的 NAT 原理上与传统 NAT 很类似，很好理解。不过也有所不同，传统 NAT 只发生在由内向外访问时，不存在由外向内访问时的 NAT。在 IP++中没有这种限制，数据包进入末单元网分两种情况：由下级单元网进入上级单元网，和由上级单元网进入下级单元网。由下向上的情况与传统 NAT 更像，而由上向下是一种全新的用法。由于反向 NAT 的使用对象是数量巨大、品种丰富的终端，有更多使用短地址及 IPv4 的需求，一方面有大量的存量设备在使用 IPv4，另外嵌入式设备往往更喜欢短地址。因此这方面有很强的现实意义。

长地址与短地址难以相互转换，因此翻译的前提是：IP++使用的也是短地址，这就要求源地址与目的地址在同一单元网内，如果这个条件能够满足当然好了，如果不满足就找 NAT 帮忙。

在很长的时间内都会存在大量的存量 IPv4 设备，如公网上的服务器、内网中的嵌入式设备等，启用了 IP++的网络要跟他们和谐相处。就可以用 NAT+翻译的方法。

3.3 双栈

这里的双栈仅仅是过渡阶段的工程实现方案。原则上 IP++和 IPv4 可以以单栈的形式实现，而且最终形态必然是单栈，也就是 IPv4 功能作为 IP++功能的附属。

根据 IP++协议规范，支持 IP++的设备必须同时支持 IPv4。因此支持 IP++的设备的协议栈中，IPv4 与 IP++两种协议的处理程序并列存在，即所谓的双栈。双栈不是互相独立的，因为 IP++地址包含了 IPv4 地址，数据包在两协议栈间有可能有交流。比如收到了 IPv4 的数据包，但应用程序是 IP++的，则需要在将数据包向上递送至传输层时切换到 IP++协议栈。对于反向的数据包，协议栈也应该知道按照相反的方向切换协议栈。因此协议栈应该保存相关状态。

前面说的是服务器程序的情况。对于客户端发出的数据包在协议栈也有可能进行双栈切换。原则上选择任一协议栈都能将数据包正确的传送到目的地。因此默认情况下不切换，也可主动进行切换。一般从 IP++切换至 IPv4 没有必要。所以可能的手动配置就是配置成从 IPv4 切换到 IP++。这时候协议栈也要保存相关状态。

双栈技术对 IP++的兼容性具有重要的意义。终端的操作系统包括协议栈升级到 IP++时，很容易遇到应用程序不能及时升级的情况，如果因此而造成应用程序不能继续使用，那是难以接受的。IP++设备中旧的只支持 IPv4 的应用程序仍能正常使用。开发新的应用程序时，只需考虑 IP++ socket 即可，不用另外花心思考虑如何兼容 IPv4，协议栈会自动处理。

4 典型应用场景

IP++的长远目标是成为网络层基础设施，全面部署于各类网络设备（路由器、终端、服务器）中。在生态构建初期，将优先聚焦于对远程互联有刚性需求的场景，以下为核心应用场景示例：

4.1 物联网

物联网的快速发展催生了海量远程互联需求（如智能家居远程控制、工业设备远程运维、环境监测数据远程上传等）。IP++的端到端通信能力可直接满足这些需求，可大幅降低物联网应用的开发、部署与运维成本，为物联网生态提供方便、高效、可靠的网络底层支撑。

4.2 地址资源扩容

IPv4 公网地址的稀缺限制了诸多应用的落地。IP++的多级变长地址架构可提供近乎无限的地址资源，运营商无需再为地址分配发愁，企业可方便廉价地实现所有终端设备的全局可访问。

4.3 计算中心网络

计算中心网络具有典型的层次化拓扑结构，与 IP++的多级变长地址架构高度契合。部署 IP++后，可简化网络配置与管理，降低建设与运维的难度。

4.4 MPTCP

MPTCP[16]通过多路径传输提升通信可靠性与带宽利用率，但传统 IP 协议的扁平化地址无法承载网络层次结构信息，导致 MPTCP 的“多路径”仅能实现多接口绑定，无法充分利用网络层次化拓扑实现路径优化。IP++的地址结构与网络层次结构相对

应，可提供精细化的路径操控能力，使 MPTCP 能够真正利用基于网络层次结构的多路径资源，充分发挥其技术优势，拓展应用范围。

5 结论

IP++通过创新的多级变长地址架构，弥补了传统 IP 协议的核心缺陷。既解决了 IPv4 地址不足与远程互联不便的问题，又规避了 IPv6 兼容性差、易用性不足的短板，使 IP 协议回归端到端的基本原则，成为真正意义上的“完整体 IP 协议”。

IP++作为网络层基础设施，涉及网络的诸多方面，本文仅对核心思想与关键机制进行阐述，更多的技术细节可参考相关技术文档。尽管 IP++的后续发展仍面临生态适配、标准制定等挑战，但从技术逻辑与实际需求来看，IP 协议向多级变长地址演进是不可阻挡的趋势。未来，随着 IP++生态的逐步完善，有望成为统一的下一代 IP 协议，推动互联网向更完善、更便捷、更具扩展性的方向发展。

参考文献

- [1] 谢希仁。计算机网络 [M]. 8 版。北京：电子工业出版社，2021.
- [2] FALL K R, STEVENS W R, WRIGHT G R. TCP/IP 详解 [M]. 吴英，译。北京：机械工业出版社，2016.
- [3] POSTEL J. RFC 791: Internet Protocol [S]. IETF, 1981.
- [4] SRISURESH P, HOLDREGE M. RFC 2663: IP Network Address Translator (NAT) Terminology and Considerations [S]. IETF, 1999.
- [5] DEERING S, HINDEN R. RFC 8200: Internet Protocol, Version 6 (IPv6) Specification[S]. IETF, 2017.
- [6] HINDEN R, DEERING S. RFC 4291: IP Version 6 Addressing Architecture[S]. IETF, 2006.
- [7] REKHTER Y, LI T, HINDEN R. RFC 4213: Basic Transition Mechanisms for IPv6 Hosts and Routers [S]. IETF, 2005.
- [8] 北京爱辟加阿加网络科技有限公司. IP++ 协议官方网站 [EB/OL]. <http://www.ippp.xyz>.
- [9] CLARK D D. Designing an Internet (Information Policy)[M]. Cambridge: MIT Press, 2018.
- [10] CLARK D D, CHAPIN L, CERF V G, et al. Towards the future internet architecture[R]. USA: Internet Architecture Board, 1991.
- [11] CHIAPPA J N. Why Topologically Sensitive Names Are Inherently Variable Length[R]. Cambridge: Massachusetts Institute of Technology, 1994.
- [12] FRANCIS P. RFC 1621: Pip Near-term Architecture[S]. IETF, 1994.

- [13] FRANCIS P. RFC 1622: Pip Header Processing[S]. IETF, 1994.
- [14] Huitema C. Flexible IP: An Adaptable IP Address Structure [R]. IETF 109, 2021.
- [15] 陈哲, 王闯, 李冠文, 等. NEW IP framework and protocol for future applications [C]//2020 IEEE/IFIP Network Operations and Management Symposium (NOMS). IEEE,2020:1-6.
- [16] FORD A, RAICIU C, HANDLEY M, et al. RFC 6824: TCP Extensions for Multipath Operation with Multiple Addresses (MPTCP)[S]. IETF, 2013.